Scaling Neural Network Verification with Tensor Parallelism and Fully Sharded Data Parallelism

Sergei Vorobyov[1] and Eugene Ilyushin[1,2]

[1] Lomonosov Moscow State University, Moscow, Russia
sergei.vorobyov01@gmail.com
[2] Central University, Moscow, Russia
eugene.ilyushin@gmail.com

Abstract. Formal neural network verification is bounded in practice by GPU memory, and it is widely assumed that the weight matrices are what has to be distributed. We test that assumption by porting two sharding strategies from large-scale model training to `auto_LiRPA` / α, β-CROWN and by measuring, rather than estimating, where the memory goes. *Tensor Parallelism* shards the weights together with the CROWN A-matrices, i.e. the specification axis: peak memory falls by 1.97× at P=2 and 3.83× at P=4 and propagation becomes 1.90× and 3.49× faster, but tightness survives only while a sharded zone contains no intermediate activation, since CROWN then falls back to IBP. *Fully Sharded Data Parallelism* shards weights alone: stored weights drop by exactly $1/P$ and IBP and CROWN bounds stay bitwise identical, yet peak memory *rises* by 26–34% and propagation costs up to 3.2× more, most of it inside collectives, and the naive baseline of staging weights from host memory beats it on both axes. We retract the 34–39% peak saving of the submitted version: it is an artefact of measuring both modes in one process. A per-node trace shows why: A-matrices are 5.6× the weights already at h=4096, and in β-CROWN+Branch-and-Bound peak memory is linear in the number of subdomains, $21.3 + 0.372\,B$ MB, of which weights are 0.13%. Sharding the subdomain axis is therefore the direction that pays, and we quantify it: 45.0% and 67.5% less memory per GPU at P=2 and P=4.

Keywords: Neural network verification · Tensor parallelism · Fully sharded data parallelism · Bound propagation · α, β-CROWN · GPU memory.

1 Introduction

Neural networks are deployed in contexts where a wrong output is not an inconvenience but a hazard: vehicle control, avionics, medical decision support. Testing cannot discharge such obligations, because a test covers one input vector rather than the admissible set [13]. Formal verification closes that gap by proving that a property holds for *every* input in a region.

Let a network define $f : \mathbb{R}^n \to \mathbb{R}^m$. Verification decides whether $\psi(f(\mathbf{x}))$ holds for all $\mathbf{x}$ in an admissible set $\mathcal{X}$; solvers work on the dual question, searching for

a counterexample $\mathbf{x} \in \mathcal{X}$ with $\neg\psi(f(\mathbf{x}))$, and report UNSAT when the property is proved and SAT with a witness otherwise [16]. The standard instance is local robustness: for a sample $\mathbf{x}_0$ of class c, every input with $\|\mathbf{x} - \mathbf{x}_0\|_p \leq \varepsilon$ must again be classified as c [15]; functional specifications over physical variables arise in control tasks such as ACAS Xu [11]. The VNN-LIB standard [10] together with ONNX fixed a common format for networks and properties and underpins the VNN-COMP competition [3,4,5]. The problem is NP-complete already for one ReLU hidden layer [7], $\exists\mathbb{R}$-complete for smooth activations [9] and undecidable for some recurrent classes [20], so a *complete* verifier needs exponential Branch-and-Bound while an *incomplete* one settles for a polynomial relaxation.

1.1 The Memory Bottleneck and the Question We Ask

The dominant practical approach is *bound propagation*. α, β-CROWN [27], winner of VNN-COMP 2021–2024, runs entirely on GPU and builds linear over-approximations of the output set. Its limit is memory: everything the propagation touches must fit on one accelerator. A single hidden layer of width h=262 144 with input dimension 4 096 already needs ≈4 GB in float32 for the weights alone.

Parameter sharding is the reflex borrowed from large-model training, and it presupposes that parameters dominate. This paper implements that reflex faithfully in a state-of-the-art verifier, measures it, and shows the presupposition to be wrong for verification: the memory sits in the tensors indexed by the specification and subdomain axes. That is a statement about where multi-GPU verification research should aim.

Contributions.

1. A measured account of GPU memory in bound propagation. A per-node trace attributes the peak of a CROWN pass, and a batch sweep gives the law $M_{\text{peak}}(B) \approx 21.3 + 0.372\,B$ MB for β-CROWN+BaB, in which weights are 0.13% (Section 4).
2. *Tensor Parallelism* for `auto_LiRPA` via custom `BoundLinearTP_Col/Row` operators, which shards the specification axis and scales both peak memory (1.97×, 3.83×) and propagation time (1.90×, 3.49×) at P=2, 4, with the tightness cost of IBP substitution quantified per sharded zone, and soundness confirmed for α-CROWN as well as CROWN (Sections 3.2, 6.1).
3. *FSDP* for bound propagation in ≈100 lines, with exactly $1/P$ stored weights, bitwise-identical IBP and CROWN bounds, and extensions to convolutions and a ViT — and with the honest verdict that it costs 26–34% *more* peak memory and up to 3.2× more time, and is beaten on both axes by host-memory staging (Sections 3.3, 6.4).
4. A retraction and its diagnosis. The 34–39% peak saving of the submitted version is an artefact of measuring two modes in one process; we exhibit the mechanism, the corrected numbers, and two memory-retention defects of our own implementation, one of which is an `AttributeError` silently swallowed by a `try/except` (Section 5).

5. A quantified target for future work: because every batch-scaled term is linear in the number of subdomains, splitting the domain axis over P GPUs predicts 45.0% and 67.5% less memory per GPU at P=2, 4, a projection validated against measurements at B/P (Section 6.5).

Our implementation, a fork of `auto_LiRPA` / α, β-CROWN, with all experiment scripts and the collated measurement records, is publicly available.[3]

2 Background

2.1 Bound-Propagation Methods

IBP [28] propagates intervals layer by layer: for $\mathbf{y} = W\mathbf{x} + \mathbf{b}$ it gives $\mathbf{l}_y = W^+\mathbf{l}_x + W^-\mathbf{u}_x + \mathbf{b}$ and $\mathbf{u}_y = W^+\mathbf{u}_x + W^-\mathbf{l}_x + \mathbf{b}$, with $W^\pm$ the positive and negative parts of W. It is cheap and loses tightness fast with depth. *CROWN* [28] instead relaxes each unstable ReLU with pre-activation $x \in [l, u]$, $l < 0 < u$, by $\lambda_l x + b_l \le \mathrm{ReLU}(x) \le \frac{u}{u-l}(x - l)$ and composes the relaxations in a backward pass,

$$A = \Lambda_L W_L \cdots \Lambda_1 W_1, \qquad \ell \;\le\; A\mathbf{x} + \mathbf{b} \;\le\; u, \tag{1}$$

with $\Lambda_i = \mathrm{diag}(\lambda_i)$. Since A is linear in the original input, cross-variable dependencies survive across all layers, which is what makes CROWN much tighter than IBP. α-*CROWN* [27] turns the fixed slopes into optimisable per-neuron parameters $\alpha_i \in [0, 1]$ and tightens the bound by gradient ascent, without branching.

Complete verification [24] adds Branch-and-Bound: when the relaxation is too loose the problem is split, e.g. by fixing a ReLU to $x \ge 0$ or $x < 0$, and bounds are recomputed per subdomain. β-CROWN encodes the split constraints as Lagrangian dual variables β inside the propagation, so thousands of subdomains are evaluated in one batched GPU pass with α and β optimised jointly; BICCOS [21] strengthens the root bound with cutting planes from the search tree, while other complete verifiers replace the search itself, by continuous relaxation [6], clause learning [12], adaptive set representations [8] or mixed-integer encodings [1].

2.2 The `auto_LiRPA` Library

All our work builds on `auto_LiRPA` [26], the library under α, β-CROWN, which expresses bound propagation over a PyTorch computational graph and derives bounds for any intermediate node of an arbitrary architecture. It supports *batch BaB*: up to hundreds of thousands of open subproblems are bounded in a single pass. Any modification must therefore keep batched processing intact and stay compatible with the traced graph, which is what makes the sharding work non-trivial.

[3] `https://github.com/vorobyov01/damdid-2026-submission`

3 Sharding Bound Propagation

3.1 Primitives, and Why Not Pipeline Parallelism

Tensor Parallelism [22] splits the matrix products inside a layer: column-parallel splits $W \in \mathbb{R}^{k\times n}$ along the output dimension into $W^{(r)} \in \mathbb{R}^{k\times n/P}$ and each GPU computes a partial output from a replicated input, while row-parallel splits along the input dimension and sums the partial products, so a Column–Row pair costs exactly one `AllReduce`. *FSDP* [19] shards every parameter tensor across P GPUs and an `AllGather` rebuilds a layer's parameter just before it is needed, releasing it just after, so at most one full weight is resident. *Pipeline parallelism* [14] does not fit CROWN: Equation 1 traverses *all* layers in one backward pass, so a pipeline would ship weight matrices between stages at every step, i.e. TP with a worse communication pattern. TP and FSDP both run the full pass on every GPU and differ in what they shard: TP the computation, FSDP only the storage.

3.2 Tensor Parallelism

We register `BoundLinearTP_Col` and `BoundLinearTP_Row`, both derived from `BoundLinear`. Each adds an `AllReduce` to `bound_backward` and registers a custom ONNX symbolic method, so that JIT tracing builds the graph *without* invoking the collective, which would otherwise raise `Tried to trace ProcessGroup`. `tp_shard_bounded_module` takes any `BoundedModule`, including one loaded from ONNX, replaces alternating `BoundLinear` pairs by their TP variants, shards the weights, and labels each Column–Row subgraph as a *sharded zone* by BFS traversal so that intermediate bounds are handled consistently.

TP shards W and the A-matrices alike, so the backward step $A \leftarrow A \cdot \Lambda_i W_i$ stays local for a Column–Row pair with no activation in between; one `AllReduce` restores the exact result. Once a ReLU sits *inside* a zone, however, CROWN needs intermediate bounds for those neurons in order to pick the slopes λ_i. Computing them exactly would mean a separate CROWN backward with inter-GPU communication at every step, which would give the memory back; the tractable alternative is IBP for those intermediate bounds. IBP discards cross-variable correlation, which produces the *wrapping effect* [17].

How fast tightness degrades. The elementary bound is worth stating, since the submitted version claimed an $O(\rho^k)$ law without proof. Let a zone consist of k linear layers with ReLUs in between, and let $w(\cdot)$ denote elementwise interval width. A linear layer maps widths by $w(W\mathbf{x}) \le |W|\, w(\mathbf{x})$ componentwise, and ReLU is non-expansive, $w(\mathrm{ReLU}(x)) \le w(x)$. Composing gives

$$\|w_{\text{out}}\|_\infty \;\le\; \left(\prod_{i=1}^{k} \big\||W_i|\big\|_\infty\right) \|w_{\text{in}}\|_\infty. \tag{2}$$

This is an upper bound on *IBP* widths, not a growth rate of the CROWN–IBP gap, and it is the only claim we make: the gap is governed by the product of

layer norms, so it compounds per zone. The measured behaviour (Table 3) is consistent with it, and $\| |W| \|_\infty$ for the three MNIST-FC models is 19.6, 20.4 and 21.1. The mechanism is elementary: for $x_1, x_2 \in [-1, 1]$ and $W = \left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$, CROWN gives $y_1 + y_2 = 2x_1 \in [-2, 2]$ exactly, while IBP treats $y_1, y_2 \in [-2, 2]$ as independent and returns $[-4, 4]$.

3.3 Fully Sharded Data Parallelism

`fsdp_shard_bounded_module` walks the graph and replaces each `BoundParams` weight by a shard along `dim=0`: $W^{(r)} \in \mathbb{R}^{k/P \times n}$ for `BoundLinear` and $W^{(r)} \in \mathbb{R}^{C_{\text{out}}/P \times C_{\text{in}} \times k_H \times k_W}$ for `BoundConv`. Dimension 0 is the output axis of both layer types, and both are linear in it, so an `AllGather` along that axis reconstructs the matrix exactly. Tensors whose leading dimension is not divisible by P are left unsharded, which at P=4 excludes only the classifier layer. Hooks `fsdp_gather_node` and `fsdp_free_node` are injected into `backward_general` (CROWN) and `IBP_general` (IBP): the matrix is assembled just before a layer computes and released just after. The whole mechanism is ≈100 lines against ≈500 for TP, because it adds no operator types and does not change the graph.

FSDP performs the same floating-point operations in the same order as the single-GPU path; only the provenance of the weight tensor differs. IBP and CROWN bounds are therefore *bitwise* identical — an exact zero, not a machine epsilon — which we confirm on eight configurations (MLPs of width 256 at depths 2, 4, 6 under IBP and CROWN, and ONNX 256×2 and 256×4 under CROWN) at P=2 and P=4. We report it as a regression guarantee rather than as a result, since it follows from the construction.

FSDP also composes with complete verification without extra algorithmic machinery: each rank runs an ordinary β-CROWN+BaB iteration once its weights have been gathered, and β is managed exactly as on one GPU. Sharding `BoundConv` over output channels required three files to change (`fsdp_utils.py` to admit `BoundConv` and to prevent double sharding, `backward_bound.py` for the gather/free in the CROWN backward, `interval_bound.py` for the free after IBP). Transformers needed three JIT fixes, because `auto_LiRPA` bakes reshape arguments in at batch size 1 while BaB later increases the batch: `BoundReshape` substitutes the actual batch size when $\prod(\text{new\_shape}) = x.\text{numel}()$, `BoundConcat` takes its tensor count from the trace, and `BoundConstantOfShape` updates its shape argument.

4 Where the Memory Actually Is

The submitted version carried two equations that treated weights and A-matrices as commensurable multiples of h^2 and predicted a 12.5% peak saving where 34% was measured; reviewers were right to object. We replace them by a decomposition that can be measured directly.

For one configuration we record two quantities: the *resident* allocation R immediately before `compute_bounds` (weight tensors, traced graph, inputs) and

the *transient* $T = M_{\text{peak}} - R$ added during the pass (A-matrices, intermediate bounds, gathered weights, collective workspaces). Both are read from `torch.cuda` counters, and R is the only part sharding can touch.

Let the network have d hidden layers of width h and let bounds be requested for N inputs. Weight storage is $S_W = 4\,(nh + (d{-}1)h^2 + hm)$ bytes. When CROWN computes intermediate bounds for a layer of width h, the specification dimension is h itself, so each live A-tensor has shape $[N, h, h]$ and a lower/upper pair costs $8Nh^2$ bytes. With c pairs live simultaneously,

$$M_{\text{peak}} \;\approx\; S_W \;+\; 8\,c\,Nh^2 \;+\; S_0, \tag{3}$$

S_0 collecting graph metadata, inputs and intermediate bounds. A per-node trace of a CROWN pass at h=4096, d=4, N=1 instantiates every term: allocation climbs monotonically from 417 MB to 1844 MB in steps of 128 MB, each step one A-pair of shape $[4096, 1, 4096]$; at the end 18 such tensors are live, i.e. c=9 and $8cNh^2 = 1152$ MB against $S_W = 204.5$ MB. *A-matrices exceed the weights by* 5.6× *already at this modest width*, and the ratio grows with N and with the specification dimension.

Equation 3 yields the ceiling that matters for FSDP. Sharding reduces S_W to S_W/P and leaves the rest untouched, so no implementation, however careful, can save more than

$$\eta_{\max} \;=\; \left(1 - \tfrac{1}{P}\right) \frac{S_W}{M_{\text{peak}}}. \tag{4}$$

Measured, $\eta_{\max}$ is 5.3% at h=4096, d=4, 5.2% at h=8192, 2.9% at d=8 and 7.9% at P=4 — single-digit numbers, against which the gathered weight and the NCCL workspace have to be paid. Section 6.4 shows that they are not paid.

In complete verification the axis is the subdomain count. In β-CROWN+BaB the tensors that dominate are indexed by the number B of subdomains in the batch: the per-neuron α slopes with shape (layers) × (specs) × (B) × (neurons), the intermediate bounds, and the A-matrices. Weights are not. A sweep over B on `mnist-net_256x6` (Table 6) gives, by least squares over six points,

$$M_{\text{peak}}(B) \;\approx\; 21.3 + 0.372\,B \quad \text{MB}, \tag{5}$$

with every batch-scaled term doubling as B doubles and the weight term pinned at 2.03 MB, i.e. 0.13% of the peak at B=4096. The submitted version attributed the whole 1.5 GB peak to α tensors; the correct statement is that α tensors are 121 MB of it (17.2%), while intermediate bounds and A-matrices take ≈ 40% each. The conclusion is unchanged and now quantified: the axis worth sharding is B.

5 One Measurement Artefact and Two Implementation Defects

5.1 Retraction: the 34–39% Peak Saving Was an Artefact

Our original harness measured both modes in one process, FSDP first, and returned the bound tensors to the caller for comparison. Under FSDP those

Table 1. The artefact at h=4096, d=4: only the transient is protocol-independent, and row 1 reproduces the submitted paper's Table 5 exactly.

Protocol	Bounds	single (MB)	FSDP=2 (MB)	"saving"
1 process, FSDP first	retained	3 799.6	2 526.9	+33.5%
1 process, single first	retained	1 944.9	4 158.7	−113.8%
1 process, FSDP first	released	1 944.9	2 526.9	−29.9%
1 process, single first	released	1 944.9	2 526.9	−29.9%
1 mode per process	**n/a**	**1 944.9**	**2 526.9**	**−29.9%**

tensors carry an autograd graph that keeps ≈1.9 GB of intermediates reachable, so the *next* measurement in that process counts them in its own peak. The single-GPU baseline was inflated, and the inflation was reported as an FSDP saving. Table 1 isolates the mechanism: whichever mode runs second is inflated, the sign of the "saving" follows the order, and the transient T — which cannot absorb another measurement's residue — is invariant at 1527.7 MB for single-GPU and 2109.7 MB for FSDP under every protocol.

All measurements in this paper now run one mode per process, with one discarded warm-up pass and three measured repeats; the standard deviation of every memory figure is 0.00 MB, so we omit error columns and report deviations for timings only. The corrected single-GPU peaks are 1.94–2.11× below the published ones, while every published FSDP figure reproduces to four significant digits, which is what pointed at the baseline rather than at the FSDP path.

5.2 Two Retention Defects in Our FSDP Implementation

The corrected numbers prompted an inventory of live CUDA tensors after one CROWN pass at h=4096, d=4, P=2, with the returned bounds deleted. It found 18 A-tensors of shape $[4096, 1, 4096]$ (1152 MB, legitimate), the three weight shards (96 MB, legitimate) and nine full $[4096, 4096]$ weight matrices (576 MB) that should have been released. Two defects were responsible.

First, `fsdp_gather_node` assembled the weight as a list of P `empty_like` buffers followed by `torch.cat`, which allocates the full weight twice per gather and records the concatenation in the autograd graph. Gathering with `all_gather_into_tensor` into a single buffer under `no_grad` removes one full copy per layer and lowers the peak by 2% (2 526.9 → 2 476.0 MB at h=4096; 9 781.7 → 9 551.8 at h=8192).

Second, and more instructive, `fsdp_free_node` released the cached bounds with `delattr` inside a `try/except AttributeError`. But `Bound.lower` and `Bound.upper` are *properties* over `_lower`/`_upper`, so `delattr` always raised, the exception was always swallowed, and every gathered weight stayed reachable through `_lower` and `_upper` for the rest of the process. The base class already provides `delete_lower_and_upper_bounds()` for exactly this purpose; calling it leaves the `BoundParams` inventory clean (shards only). We report the pattern because a silently swallowed `AttributeError` around a property is a defect that

Table 2. TP scaling. Peak memory and propagation time both scale close to linearly; collectives are seven calls and under 1% of propagation. Single-GPU at H=131 072 equals the submitted TP=2 per-rank figure at H=262 144, so the TP memory model is exact.

Mode	Peak (MB)	Reduction	Build (ms)	Propagation (ms)	Speed-up	Comm (ms)
Single	13 513.2	1.00×	4 279.5	546.6	1.00×	0.0
TP=2	6 856.7	1.97×	22 679.8	287.7	1.90×	89.2
TP=4	3 528.4	3.83×	11 392.1	156.5	3.49×	139.9

no bound-value test can detect — bounds stayed bitwise identical throughout — and only a memory inventory exposes it.

Neither fix changes the verdict: with both applied the FSDP peak is still above the single-GPU peak in every configuration, because Equation 4 caps the achievable saving at a few percent.

6 Experiments

Setup. All measurements new in this revision ran on 4× NVIDIA RTX PRO 4000 Blackwell (24 GB, PCIe, no NVLink, driver 580.159.04), 48 vCPU, PyTorch 2.8.0+cu128, CUDA 12.8, NCCL with `NCCL_P2P_DISABLE=1`, launched by `torchrun`; one mode per process, seed 42, one discarded warm-up and three measured repeats. The submitted version used 2× NVIDIA A40 (48 GB), and every FSDP figure from it reproduces here to four significant digits, confirming that peak *allocated* memory follows tensor shapes rather than the device. The ViT and CIFAR-100 results of Section 6.6 are the A40 measurements, taken one mode per process and hence unaffected by Section 5.

6.1 Tensor Parallelism: Memory and Time

The model is $f(\mathbf{x}) = W_2\,\mathrm{ReLU}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$ with W_1 Column-parallel and W_2 Row-parallel following the Megatron-LM pattern [22]; D=4096, H=131 072, N=2048, ε=0.01, CROWN. H is half of the submitted version's 262 144 so that the single-GPU baseline fits in 24 GB. Graph construction is timed separately from propagation, because the custom TP operators make JIT tracing markedly more expensive and that cost is paid once per model, not once per query — conflating the two, as we initially did, makes TP look 4.8× slower than the baseline when in fact its propagation is faster.

Table 2 is the strongest positive result in this paper and it comes with a caveat that the submitted version omitted. On this very model the TP bounds are useless: $\sum lb = -3.35 \times 10^8$ against -6.65×10^3 on one GPU, because the ReLU lies *between* the Column and the Row layer, i.e. inside the sharded zone, and its intermediate bounds fall back to IBP (Section 3.2). The memory and time scaling of TP is real, but it is only usable where a sharded zone contains no intermediate activation.

Table 3. TP=2 deviation from the single-GPU reference. One Column–Row pair with no activation inside the zone agrees to machine precision; each further zone compounds the IBP substitution, consistently with Equation 2. Soundness holds throughout, and cross-rank deviation is zero everywhere.

Model	Zones	$\| \lvert W \rvert \|_\infty$	$\lvert \Delta lb \rvert$	$\lvert \Delta ub \rvert$	Sound
PyTorch 256×2	1	—	3×10^{-8}	3×10^{-8}	✓
PyTorch 256×4	2	—	2.51	2.49	✓
ONNX 256×2	1	19.6	6×10^{-8}	6×10^{-8}	✓
ONNX 256×4	2	20.4	5.64	3.98	✓
ONNX 256×6	3	21.1	792	750	✓

6.2 Tensor Parallelism: Soundness and Tightness

Bounds from single-GPU CROWN were compared against TP=2 on PyTorch MLPs and on VNN-COMP 2022 MNIST-FC ONNX models [3] (ε=0.02, L_∞), checking the deviations $|\Delta lb|$, $|\Delta ub|$, one-sided soundness ($lb_{\text{tp}} \leq lb_{\text{ref}}$, $ub_{\text{tp}} \geq ub_{\text{ref}}$) and cross-rank agreement.

Reviewers asked whether soundness had been established for α-CROWN and not only for CROWN. It has: against a single-GPU α-CROWN reference, TP=2 agrees to $|\Delta lb|_{\max} = 4.47 \times 10^{-8}$ (mean 2.14×10^{-8}, relative 3.64×10^{-7}), passing at tolerance 10^{-4}, and the TP lower bound came out 8.0×10^{-4} *tighter*, which is the expected direction for a sound over-approximation. `tp_shard_bounded_module` applied to all three MNIST-FC ONNX models without manual intervention, traversing the graph in topological order, replacing alternating `BoundLinear` pairs, re-running a forward pass to refresh metadata and labelling the zones; all three pass the checks above.

6.3 FSDP: Exactness, and Its Limits

On eight configurations (MLPs of width 256 at depths 2, 4, 6 under IBP and CROWN; ONNX 256×2 and 256×4 under CROWN) FSDP reproduces the single-GPU bounds with $|\Delta lb| = |\Delta ub| = 0.00$, an exact zero, at both P=2 and P=4, before and after the fixes of Section 5. One qualification is due, and tracking it down was worth the effort. In the *optimised* α/β path a distributed run does not match a plain single-GPU run bit for bit: on `mnist-net_256x6` at ε=0.05, three of nine specification bounds differ, by at most 6.10×10^{-5} absolute and 1.17×10^{-7} relative. Sharding is not the cause. Our distributed launcher disables the TorchScript fusers, a workaround for a strides assertion that fires only on non-primary GPUs, and reproducing exactly that fuser configuration on a single GPU with no process group at all reproduces the distributed bounds digit for digit, while the untouched single-GPU run keeps its own values. The deviation is the price of the fuser workaround, not of weight sharding, so FSDP is exact in the sense we claimed — but a distributed run of α, β-CROWN is not bit-identical to a single-GPU one for reasons that have nothing to do with sharding, and that is worth knowing for anyone comparing multi-GPU verification results.

Table 4. FSDP against single-GPU, one mode per process, after both fixes of Section 5. Stored weights fall by exactly $1/P$ while peak memory rises; P=4 is indistinguishable from P=2 because the term it shards is not the term that dominates.

	Weights on GPU (MB)			Peak (MB)			
Model	single	FSDP=2	offload	single	FSDP=2	FSDP=4	offload
h=1024, d=4	15.1	7.6	0	149.6	200.6	200.6	184.6
h=4096, d=4	204.5	102.3	0	1 944.9	2 476.0	2 475.9	2 271.6
h=8192, d=4	792.9	396.5	0	7 593.9	9 551.8	9 551.5	8 759.0
h=4096, d=8	460.5	230.3	0	7 934.0	10 181.1	10 180.9	9 720.7

Table 5. Bound-propagation time (ms). Parentheses: for FSDP, time inside `AllGather` (ms); for offloading, staged host-to-device volume (MB), whose copy time is contained in the total. The naive baseline is cheaper than sharding on every configuration.

Model	single	FSDP=2	FSDP=4	offload
h=1024, d=4	23.6	30.1 (17.3)	54.6 (43.7)	21.5 (51 MB)
h=4096, d=4	74.7	236.6 (166.7)	286.7 (217.4)	103.3 (637 MB)
h=8192, d=4	418.1	983.3 (583.5)	1 128.1 (733.7)	528.4 (2 427 MB)
h=4096, d=8	381.1	867.4 (510.8)	1 003.0 (653.4)	477.6 (2 350 MB)

6.4 FSDP: Corrected Memory and Time

Tables 4 and 5 replace Table 4 of the submitted version. Stored weights do drop by exactly $1/P$, as claimed, and that claim was never in doubt; what the corrected protocol removes is the inference that peak memory drops with it. Peak memory instead rises by 26–34%, and the ceiling of Equation 4 explains why this was never avoidable: at h=4096, d=4 the whole weight term is 10.5% of the peak, so P=2 could have saved at most 5.3% of it, less than the cost of staging a full weight. Going from P=2 to P=4 moves the peak by 0.1 MB out of 2 476 while adding 50 ms of collectives.

Propagation is 2.3–3.2× slower under FSDP=2 at $h \geq 4096$ (1.3× at h=1024, the smallest workload), and 57–70% of the wall-clock is spent inside `AllGather`: 166.7 of 236.6 ms at h=4096, 583.5 of 983.3 ms at h=8192. On a PCIe machine without NVLink that overhead is structural, since one full weight per layer crosses the bus at every intermediate-bound computation — 19 collectives and 842 MB moved for a d=4 pass at h=4096.

The naive baseline is better. Reviewers asked how sharding compares with simply keeping the weights in host memory and staging them per layer. We implemented that baseline by reusing the same hook sites with a host-to-device copy in place of the collective, which leaves *no* weight bytes resident on the GPU — a strictly larger reduction than FSDP's $1/P$ — and produces bounds identical to the single-GPU reference. It wins on both axes: at h=4096, d=4 it needs 2 271.6 MB against FSDP's 2 476.0 and 103.3 ms against 236.6, and the pattern holds at

Table 6. Where BaB memory goes, by number of subdomains B (single GPU, MB). Every batch-scaled term doubles with B; the weight term does not move.

B	α	β	Interm. bounds	A-matrices	Weights	Peak
128	3.79	0.01	9.06	9.08	2.03	68.73
256	7.58	0.02	18.13	18.16	2.03	116.24
512	15.16	0.05	36.26	36.33	2.03	211.93
1 024	30.31	0.10	72.51	72.66	2.03	402.52
2 048	60.62	0.20	145.03	145.32	2.03	780.72
4 096	121.25	0.40	290.06	290.64	2.03	1 543.61

every size (Tables 4, 5). Activation checkpointing, the other naive alternative, does not apply at all: the CROWN backward stores no forward activations to recompute, and its A-matrices are live because they are still needed.

That the crude mechanism beats the sophisticated one is the clearest statement of this paper's finding. Neither mechanism can win, because both pay to stage a full weight into a pass whose peak is set by tensors they never touch: at h=4096 the staged copies that `auto_LiRPA` retains past the layer that needs them cost 531 MB, more than twice the 204.5 MB of weights either mechanism removes.

6.5 Complete Verification: the Memory Is in the Subdomains

`mnist-net_256x6` [3], ε=0.05, exactly 10 BaB rounds, `bab.force_synchronous` enabled so that single-GPU and FSDP traverse an identical workload (fixed batch, no auto-enlarge, no early stop, no pruning in iteration).

Two things follow. First, FSDP cannot help here and does not: at B=512 the peak is 211.9 MB on one GPU and 214.9 MB per rank at both P=2 and P=4, because there are 2.03 MB of weights to shard and the `AllGather` buffers cost more than that. This confirms the submitted version's observation, with the correction that α tensors are 17.2% of the peak rather than all of it, intermediate bounds and A-matrices taking about 40% each.

Second, Equation 5 makes the pay-off of sharding the subdomain axis calculable instead of speculative. Splitting a global batch B across P GPUs leaves each with B/P subdomains and hence $21.3 + 0.372\,B/P$ MB, i.e. 45.0% less memory per GPU at P=2 and 67.5% less at P=4 for B=512 (49.3% and 73.9% at B=4096). The projection is not an extrapolation: the single-GPU run at B/P subdomains *is* the per-GPU workload of the split, so the measured row at B=256 (116.24 MB) is the predicted P=2 per-GPU peak for B=512 (116.5 MB), agreeing to 0.2%. What such a split still has to pay is the gather of the per-domain results, and the implementation went two steps further for this revision. First, the scatter belongs *before* `build_history_and_set_bounds`: our earlier attempt sliced the domains after that call, when the model state had already been sized for the full batch. Moving the slice ahead of it lets each rank build its own state and propagate its own chunk, which the round-level output confirms. Second, domain-parallelism does not compose with FSDP, whose per-layer `AllGather`

Table 7. Optimiser and search settings. All configuration files are in the repository; the fair-comparison configs additionally set `force_synchronous`, `auto_enlarge_batch_size: false`, `pruning_in_iteration: false` and `pgd_order: skip`.

Setting	MNIST-FC 256×6	ViT `pgd_2_3_16`	CIFAR-100 ResNet
α-CROWN lr / iters	0.1 / 20	0.5 / 50	0.25 / 20
β-CROWN lr_α	0.1	0.05	0.1
β-CROWN lr_β / iters	0.03 / 20	0.05 / 10	0.2 / 10
Batch	128–4 096	32	64
BaB `max_iterations`	10	10	5
Branching	default	`reduceop: min`	`kfsb`, 7 cand.
ε	0.05	benchmark	0.0039

is interleaved with data-dependent control flow: once two ranks hold different subdomains their collective sequences diverge and the run spins in NCCL. Disabling weight sharding removes that and is the right design here anyway, the weights being 2.03 MB. What still stalls is the gather of the per-rank results, so we report the projection rather than a measurement and name that gather as the remaining task.

6.6 Convolutions and Transformers: Correctness Checks

These runs establish that the sharding paths work beyond MLPs; we label them correctness checks rather than benefits, since none saves memory. On the VNN-COMP'23 ViT `pgd_2_3_16` (2 blocks, 3 heads, 0.3 MB of weights) [4], after the three JIT fixes of Section 3.3, FSDP=2 completes 78 BaB domains over 10 rounds at 1 020.1 MB per rank with $|\Delta lb| = 0.00$ at printed precision — to our knowledge the first Transformer verified in α, β-CROWN under weight sharding. On CIFAR-100 ResNets [5] with `BoundConv` sharding (batch 64, `max_iterations` 5), ResNet-med runs 130 domains over 5 rounds at 930 MB per rank against 960 MB on one GPU and exhausts its budget (*unknown*), while ResNet-large (≈95 MB of weights) is proved **unsat** by α-CROWN at initialisation at 4 641.5 MB per rank against 4 610.6 MB on one GPU. That *unsat* is a correctness check, not a benefit: no BaB ran, and the FSDP peak is the higher of the two, its `AllGather` overhead exceeding what sharding 95 MB of weights returns when the α tensors alone exceed 4 GB.

6.7 Reproducibility

Seed 42 throughout; every memory figure is the mean of three repeats after one warm-up and its standard deviation is 0.00 MB in every configuration, which is why the tables carry no error columns. Timing deviations are below 5% in every configuration at $h \geq 4096$; at h=1024, where a pass lasts tens of milliseconds, jitter reaches 24% (single GPU) and 43% (FSDP=4). The scripts behind every table and the collated measurement records (`experiments/analysis/RESULTS.md`) are in the repository.

7 Related Work

Bound propagation. CROWN [28] introduced the backward linear relaxation, α-CROWN [27] and β-CROWN [24] added optimisable slopes and dual encoding of branching constraints, DeepPoly [23] works in a polyhedral abstract domain and OSIP [2] optimises symbolic interval propagation.

Parallel and multi-GPU verification. Two lines exist, and neither addresses per-GPU parameter storage. Search-space parallelism distributes the Branch-and-Bound tree: Marabou's Split-and-Conquer [25] partitions the input domain across CPU workers with iterative deepening, and β-CROWN [24] parallelises *within* one GPU by batching subdomains. Kernel-level parallelism ports a fixed abstract domain to the GPU, as GPUPoly [18] does for DeepPoly. These scale the *number* of subproblems, whereas we attacked the memory of a single one; our measurements say the subproblem batch is exactly where that memory is, which puts domain-level splitting and memory reduction on one axis rather than on orthogonal ones.

Sharding for training. Megatron-LM [22] introduced intra-layer tensor parallelism, ZeRO [19] shards parameters, gradients and optimiser states, and GPipe [14] pipelines layer groups with micro-batches. To our knowledge neither had been applied to bound-propagation verification, which is what makes the negative result worth recording: a training step is dominated by parameters and optimiser state, a verification pass by tensors indexed by specifications and subdomains.

8 Conclusion

Porting parameter sharding from large-model training to formal verification is cheap in engineering terms and instructive in its outcome. TP shards the specification axis and delivers close to linear scaling of both peak memory (1.97×, 3.83×) and propagation time (1.90×, 3.49×) at $P{=}2, 4$, but only for sharded zones that contain no intermediate activation, since CROWN otherwise degrades to IBP and the bounds become unusable. FSDP shards the parameter axis, keeps IBP and CROWN bounds bitwise identical and reduces stored weights by exactly $1/P$, yet raises peak memory by 26–34% and propagation time by up to 3.2×, and is beaten on both axes by the naive baseline of staging weights from host memory. We retract the 34–39% peak saving of the submitted version, which came from measuring two modes in one process, and we report the ceiling that made such a saving impossible in the first place: the weight term is 6–11% of the peak, so sharding half of it could never return more than a few percent.

The finding we would carry forward is the measurement, not either mechanism. In incomplete verification A-matrices exceed weights by 5.6× at $h{=}4096$; in complete verification peak memory is $21.3 + 0.372\,B$ MB in the number of subdomains, of which weights are 0.13%. Multi-GPU verification should therefore shard the subdomain axis, for which we give a validated projection of 45.0% and

67.5% less memory per GPU at $P{=}2, 4$; the scatter itself already runs correctly per rank, and the remaining implementation task is the gather of per-rank results. Sharding the specification axis without the IBP fallback — exact intermediate bounds for neurons inside a sharded zone — is the harder and more valuable open problem.

Acknowledgments. The authors thank the `auto_LiRPA` / α, β-CROWN team for the open-source codebase that made this work possible, and the reviewers of DAMDID 2026, whose objection to the memory analysis led directly to Sections 4 and 5.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Evaluating robustness of neural networks with mixed integer programming (2017), https://arxiv.org/abs/1711.07356
2. Optimized symbolic interval propagation for neural network verification (2022), https://arxiv.org/abs/2212.08567
3. The third international verification of neural networks competition (vnn-comp 2022): Summary and results (2022), https://arxiv.org/abs/2212.10376
4. The fourth international verification of neural networks competition (vnn-comp 2023): Summary and results (2023), https://arxiv.org/abs/2312.16760
5. The fifth international verification of neural networks competition (vnn-comp 2024): Summary and results (2024), https://arxiv.org/abs/2412.19985
6. Marabou 2.0: A versatile formal analyzer of neural networks (2024), https://arxiv.org/html/2401.14461v1
7. Robustness verification in neural networks (2024), https://arxiv.org/abs/2403.13441
8. Bak, S.: nnenum: Verification of ReLU neural networks with optimized abstraction refinement (2021), https://www.researchgate.net/publication/351683889_nnenum_Verification_of_ReLU_Neural_Networks_with_Optimized_Abstraction_Refinement
9. Bertschinger, D., Hertrich, C., Jungeblut, P., Miltzow, T., Weber, S.: Training neural networks is $\exists\mathbb{R}$-complete. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 34 (2021), https://proceedings.neurips.cc/paper_files/paper/2021/file/9813b270ed0288e7c0388f0fd4ec68f5-Paper.pdf
10. Demarchi, S., Guidotti, D., Pulina, L., Tacchella, A.: VNN-LIB: The verification of neural networks library (2021), https://www.vnnlib.org/
11. Demarchi, S., Guidotti, D., Pulina, L., Tacchella, A.: Supporting standardization of neural networks verification with vnn-lib and coconet. In: Proceedings of the 6th Workshop on Formal Methods for ML-Enabled Autonomous Systems (FoMLAS 2023). Kalpa Publications in Computing, vol. 16, pp. 47–58 (2023), https://easychair.org/publications/paper/Qgdn/open
12. Duong, H., Nguyen, T., Dwyer, M.: NeuralSAT: A DPLL(T)-based framework for verifying deep neural networks (2023), https://github.com/dynaroars/neuralsat
13. Guidotti, D.: Enhancing neural networks through formal verification. In: Proceedings of the AI*IA 2019 Doctoral Consortium (AI*IA 2019 DC). CEUR Workshop Proceedings, vol. 2495, pp. 107–112. CEUR-WS.org (2019), https://ceur-ws.org/Vol-2495/paper13.pdf

14. Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M.X., Lee, H., Ngiam, J., Le, Q.V., Wu, Y., Chen, Z.: Gpipe: Efficient training of giant neural networks using pipeline parallelism. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 32 (2019), https://arxiv.org/abs/1811.06965
15. Liu, J., Chen, L., Miné, A., Wang, J.: Input validation for neural networks via runtime local robustness verification. CoRR **abs/2002.03339** (2020), https://arxiv.org/abs/2002.03339
16. Marzari, L., Mastroeni, I., Farinelli, A.: Advancing neural network verification through hierarchical safety abstract interpretation (2025), https://arxiv.org/abs/2505.05235
17. Moore, R.E., Kearfott, R.B., Cloud, M.J.: Introduction to Interval Analysis. SIAM (2009)
18. Müller, C., Makarchuk, G., Singh, G., Püschel, M., Vechev, M.: Scaling polyhedral neural network verification on GPUs (2021), https://files.sri.inf.ethz.ch/website/papers/mller2021neural.pdf
19. Rajbhandari, S., Rasley, J., Ruwase, O., He, Y.: Zero: Memory optimizations toward training trillion parameter models (2020), https://arxiv.org/abs/1910.02054
20. Ruan, W., Huang, X., Kwiatkowska, M.: Reachability analysis of deep neural networks with provable guarantees. In: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI) (2018), https://www.ijcai.org/proceedings/2018/0368.pdf
21. Shi, Z., Jin, Q., Kolter, J.Z., Jana, S., Hsieh, C.J., Zhang, H.: Scalable neural network verification with branch-and-bound inferred cutting planes. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 37 (2024), https://proceedings.neurips.cc/paper_files/paper/2024/file/33d93e4dc57453e7667b20f62e7c0681-Paper-Conference.pdf
22. Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., Catanzaro, B.: Megatron-lm: Training multi-billion parameter language models using model parallelism (2020), https://arxiv.org/abs/1909.08053
23. Singh, G., Gehr, T., Püschel, M., Vechev, M.: An abstract domain for certifying neural networks (2019), https://ggndpsngh.github.io/files/DeepPoly.pdf
24. Wang, S., Zhang, H., Xu, K., Lin, X., Jana, S., Hsieh, C.J., Kolter, J.Z.: Beta-CROWN: Efficient bound propagation with per-neuron split constraints for neural network robustness verification. In: Advances in Neural Information Processing Systems (NeurIPS) (2021), https://arxiv.org/abs/2103.06624
25. Wu, H., Ozdemir, A., Zeljić, A., Julian, K., Irfan, A., Gopinath, D., Fouladi, S., Katz, G., Păsăreanu, C., Barrett, C.: Parallelization techniques for verifying neural networks. In: Formal Methods in Computer Aided Design (FMCAD). pp. 128–137 (2020)
26. Xu, K., Shi, Z., Zhang, H., Wang, Y., Chang, K.W., Huang, M., Kailkhura, B., Lin, X., Hsieh, C.J.: Automatic perturbation analysis for scalable certified robustness and beyond (2020), https://github.com/Verified-Intelligence/auto_LiRPA
27. Zhang, H., Wang, S., Xu, K., Li, L., Li, B., Jana, S., Hsieh, C.J., Kolter, J.Z.: alpha-beta-CROWN: A complete and efficient bound propagation based neural network verifier (vnn-comp 2021–2024) (2023), https://github.com/Verified-Intelligence/alpha-beta-CROWN_vnncomp23
28. Zhang, H., Weng, T.W., Chen, P.Y., Hsieh, C.J., Daniel, L.: Efficient neural network robustness certification with general activation functions (2018), https://arxiv.org/abs/1811.00866

Appendix: Reviewers' Remarks and Our Responses

This appendix is supplied for the meta-review of the revised submission and is intended to be removed from the camera-ready version, which keeps the main text within the page limit.

The reviews prompted us to re-run every measurement in this paper. That re-run invalidated one of our three headline claims, so the revision is substantial: Sections 4 and 5 are new, Section 6 is re-measured, and the abstract, contributions and conclusion are rewritten. We are grateful for the objections that led there.

Reviewer 1

W1. "Framing contradicts the main finding. The abstract foregrounds FSDP as meaningful peak savings, hiding that this holds only in incomplete mode on wide MLPs. The α-tensor finding should be the headline."
Accepted, and it turned out to be worse than described: the peak saving does not hold in incomplete mode either. Re-measuring with one mode per process shows FSDP *raising* peak memory by 26–34% (Table 4); the 34–39% figure was a measurement artefact, retracted and diagnosed in Section 5. The paper is now organised around where the memory is: Section 4 derives and measures the decomposition, the abstract leads with it, and contributions 1 and 5 state it.
W2. "No scaling and no timing. All 8 experiments use only P=2, and only memory is ever measured, leaving the AllReduce/AllGather overhead unquantified (critical on the PCIe Server B)."
Addressed. Every sharding experiment now reports P=2 and P=4 (Tables 2, 4, and B=512 under FSDP=4 in Section 6.5), and every run reports wall-clock with the collective time separated out by CUDA events. The machine is PCIe without NVLink, as the reviewer anticipated: 57–70% of FSDP propagation time is inside `AllGather`. Timing also corrected a mistake of ours: separating one-off graph construction from propagation turned TP from "4.8× slower" into 1.90× and 3.49× *faster* at P=2, 4 (Section 6.1).
W3a. "The memory formulas (2)–(3) are trivial identities that contradict the measurements (they predict 12.5% vs the measured 34–39%, a 3× gap left unexplained)."
The reviewer was right, and following the contradiction resolved it in the unexpected direction: the measurements were wrong, not the formulas. The formulas are gone, replaced by an operational decomposition into resident and transient allocation (Equation 3) that is read from `torch.cuda` counters, instantiated by a per-node trace, and closed with the ceiling of Equation 4, which shows that no FSDP implementation could have saved more than 3–8% of peak.
W3b. "The $O(\rho^k)$ theorem is unproven and misapplied (the classical result assumes identical weights and no ReLU, and ρ is never reported)."
Accepted; the claim is withdrawn. Section 3.2 now proves the elementary bound $\|w_{\text{out}}\|_\infty \le \prod_i \| \, |W_i| \, \|_\infty \cdot \|w_{\text{in}}\|_\infty$ (Equation 2), using only that a linear layer

maps widths by $|W|$ and that ReLU is non-expansive, states explicitly that it bounds IBP widths rather than the CROWN–IBP gap, and reports $\| |W| \|_\infty$ for the three MNIST-FC models (Table 3).

W3c. "Bitwise-identical FSDP, although correct, is trivial by construction (an 8-row table of zeros presented as a result)."
Accepted. The table is gone; the property is stated in two sentences and labelled a regression guarantee rather than a result (Section 3.3). Re-checking it also uncovered something we had missed and now report: in the optimised α/β path a distributed run agrees with a plain single-GPU run only to 1.17×10^{-7} relative — and the cause is not sharding but the TorchScript fusers, which our launcher disables on multi-GPU runs. Applying that same fuser configuration on a single GPU reproduces the distributed bounds digit for digit.

W4. "Reproducibility gaps. The α/β-optimizer hyperparameters are never reported, and there are no seeds or run variances."
Addressed: Table 7 lists learning rates, iteration counts, batch sizes, branching settings and ε per benchmark; seed 42 throughout; one discarded warm-up and three measured repeats per configuration, with standard deviations reported (0.00 MB for every memory figure, under 1.2% for timings except the smallest FSDP=4 workload).

W5. "Positioning gaps: domain-parallel BaB stated with no citation; the ResNet-large unsat demonstrates correctness not value; TP soundness shown only for CROWN; TP usable for at most 2 zones; no external baselines."
All five addressed. Parallel verification is now cited and discussed (Section 7): Split-and-Conquer in Marabou [25], batched-subdomain parallelism in β-CROWN [24] and GPU kernel-level work in GPUPoly [18]. The ResNet-large *unsat* is relabelled a correctness check in Section 6.6, stating that no BaB ran and that the FSDP peak is the higher of the two. TP soundness under α-CROWN is measured (Section 6.2): $|\Delta lb|_{\max} = 4.47 \times 10^{-8}$, sound, bound 8.0×10^{-4} tighter. The usability limit is now stated in the text and in the conclusion instead of being implied by a table — and stated more sharply than the reviewer put it: machine precision holds only for zones with no internal activation, and the deviation compounds per zone (2.5 at two zones, 792 at three, Table 3). On external baselines: the single-GPU α, β-CROWN path is the VNN-COMP-winning implementation and is the meaningful baseline for a memory claim, since no other verifier shards weights across GPUs; we also address the naive alternatives raised by Reviewer 2 below.

Suggestions. (1) The memory-decomposition finding is now the headline of the abstract, contributions and conclusion. (2) P=4 is included throughout. (3) Hyperparameters, seeds and standard deviations are reported, and the $O(\rho^k)$ claim is replaced by a proved bound. (4) Parallel verification is cited, and the ResNet-large result is qualified as a correctness check.

Reviewer 2

"Experiments only with P=2; scalability to more GPUs not investigated." Addressed: P=4 in every re-measured experiment (the ViT and CIFAR-100 correctness checks remain at P=2), and it is itself a finding that FSDP peak memory is unchanged from P=2 to P=4 (2 476.0 vs 2 475.9 MB), because the term it shards is not the term that dominates, whereas TP continues to scale ($1.97\times \to 3.83\times$).
"No data on execution time or synchronisation overhead." Addressed: wall-clock and collective time for every configuration, with 57–70% of FSDP propagation inside `AllGather` on a PCIe machine.
"The main conclusion about α tensors is not accompanied by proposals for optimising them." Addressed and made quantitative. The batch sweep gives $M_{\text{peak}}(B) \approx 21.3 + 0.372\,B$ MB (Equation 5), so splitting the subdomain axis over P GPUs is predicted to cut per-GPU memory by 45.0% at P=2 and 67.5% at P=4; the projection is validated against the measured single-GPU point at B/P (0.2% agreement). Section 6.5 also reports how far we got with the implementation: the scatter has to precede `build_history_and_set_bounds`, and domain-parallelism cannot be combined with FSDP because interleaved per-layer collectives desynchronise once ranks hold different subdomains. Per-rank propagation now runs correctly; the gather of per-rank results is what remains.
"No comparison with naive approaches (checkpointing, CPU-offloading)." Addressed by implementation, and the outcome changed our conclusion. We first answered this by argument and got it wrong: we reasoned that host offloading was bounded by the same ceiling as FSDP, whereas it removes the *entire* weight term from GPU residency rather than the $(1 - 1/P)$ fraction. So we implemented it — the same hook sites with a host-to-device copy in place of the collective — and measured it. It beats FSDP on both axes at every size: 2 271.6 MB against 2 476.0 and 103.3 ms against 236.6 at h=4096, d=4, with bounds identical to the single-GPU reference (Tables 4 and 5). Activation checkpointing, by contrast, does not apply at all: the CROWN backward stores no forward activations to recompute, and its A-matrices are live because they are still needed, not because they are cached. We are grateful for the remark: the crude baseline outperforming the sophisticated mechanism is now the sharpest formulation of the paper's result.

Reviewer 3

The reviewer found the work solid and well structured and identified the α-tensor observation as its contribution, noting that a complete solution to the memory problem is left to future work. We agree, and the revision makes that observation the organising claim rather than a closing remark: Section 4 measures the decomposition, Equation 5 turns it into a law, and Section 6.5 converts the future work into a validated quantitative target. The reviewer's summary that "either coarse approximations are used, which is inadmissible for verification, or the memory saving is small" is precisely the trade-off that Tables 2 and 4 now make explicit.